

Multi-Asset Portfolio Construction Platform

Deep Dive: Modules 8–14

Portfolio Theory, Risk, Performance & Optimization

CONCEPTS • MATH WALKTHROUGH • CODE EXPLANATION

Module 8 — Portfolio Theory
Module 9 — Portfolio Risk Engine
Module 10 — Performance Analytics
Module 11 — Portfolio Construction
Module 12 — Mean-Variance Optimization
Module 13 — Portfolio Constraints
Module 14 — Advanced Portfolio Optimization

Module 8 — Portfolio Theory

Every construction, risk, and optimization step in this project rests on the formulas in this module. Get these right and everything downstream makes sense.

Part 1 — Concepts

Diversification

Combining assets that don't move in lockstep reduces overall portfolio risk without a proportional loss in expected return. This is qualitative until we build the math to prove it.

Expected Return, Variance, Covariance, Correlation

- **Variance** — how much a single asset's returns spread out around their mean. The basic measure of that asset's own risk.
- **Covariance** — how two assets' returns move together. Positive = same direction, negative = opposite.
- **Correlation** — covariance normalized to always fall between -1 and +1, so it's comparable across any pair of assets.

The Efficient Frontier

If you compute risk and return for every possible weight combination, some combinations are objectively better than others: for a given risk level, there's a maximum achievable return. The curve connecting these best combinations is the Efficient Frontier. Anything below/right of it is "dominated" — a better option exists at equal or lower risk.

Risk-Free Rate & Sharpe Ratio

The risk-free rate is the theoretical return of a zero-risk investment — the reference point for asking "was taking this risk worth it?" The Sharpe Ratio answers exactly that: excess return earned per unit of risk taken.

Part 2 — Math Walkthrough

Portfolio Expected Return

$$E(R_p) = \sum w_i \cdot E(R_i)$$

Portfolio return is the **weighted average** of individual asset returns. This only works correctly with *simple* returns (not log returns), because only simple returns aggregate linearly by weight.

Portfolio Variance — the key diversification formula

For two assets:

$$\sigma_p^2 = w_1^2 \sigma_1^2 + w_2^2 \sigma_2^2 + 2w_1 w_2 \sigma_{12}$$

Generalized to n assets using matrix notation:

$$\sigma_p^2 = \mathbf{w}^T \Sigma \mathbf{w}$$

where $\mathbf{w}$ is the weight vector and Σ is the $n \times n$ covariance matrix. **This formula is the entire mathematical justification for diversification.** If σ_{12} is negative, the cross-term *subtracts* from total variance — the portfolio can have genuinely lower risk than a naive weighted average of individual risks would suggest.

Worked proof (equal-weight, 9-asset universe):

Naive weighted-average volatility (no diversification): 17.90%

Actual portfolio volatility (with diversification): 11.97%

Diversification benefit: 5.93 percentage points — a ~33% relative risk reduction, achieved with nothing more than equal weighting across imperfectly-correlated assets.

Sharpe Ratio

$$\text{Sharpe} = (E(R_p) - R_f) / \sigma_p$$

Why subtract R_f at all, rather than just using $E(R_p)/\sigma_p$? Because investors should only be rewarded for taking *risk*. An 8% return means something very different depending on whether the risk-free rate is 1% or 5% — Sharpe isolates the excess return actually earned for bearing risk.

Part 3 — Code Explanation

src/portfolio/portfolio_theory.py

```
def portfolio_variance(weights, covariance_matrix):
    return np.dot(weights.T, np.dot(covariance_matrix, weights))
```

`np.dot(covariance_matrix, weights)` computes $\Sigma \mathbf{w}$ first — a vector where each entry says "how this asset co-moves with the whole weighted portfolio." The outer `np.dot(weights.T, ...)` collapses that into one number: portfolio variance. This is the matrix form doing, in one line, what the two-asset formula does by hand — generalized automatically to any number of assets.

```
def sharpe_ratio(portfolio_return, portfolio_vol, risk_free_rate=0.0):
    return (portfolio_return - risk_free_rate) / portfolio_vol
```

Defaults `risk_free_rate` to 0.0 rather than hardcoding an assumed "typical" rate — an unexplained hardcoded assumption is exactly what Rule 9 (no hidden assumptions) warns against. The real rate is supplied explicitly wherever this function is called.

Module 9 — Portfolio Risk Engine

Volatility alone treats an unexpected gain the same as an unexpected loss. This module builds the fuller risk picture — downside risk, tail risk, and benchmark-relative risk — that real risk managers actually report.

Part 1 — Concepts

Downside Volatility

Standard volatility counts both up and down swings equally. Downside volatility only counts returns below zero — capturing the risk investors actually fear, not the variability they welcome.

Maximum Drawdown

The largest peak-to-trough decline in portfolio value. Unlike volatility (a statistical abstraction), this is the number an investor actually *feels* — "how much did I lose from my best point before recovering."

VaR and CVaR

- **Historical VaR** — reads the actual empirical return distribution directly. No distributional assumption, but limited by whatever extremes did or didn't occur historically.
- **Parametric VaR** — assumes returns are normally distributed, using only mean and standard deviation. Fast, but blind to fat tails.
- **CVaR (Expected Shortfall)** — the *average* loss within the worst tail, not just the threshold. Always $\geq$ VaR at the same confidence level.

Why we skip Parametric CVaR: it inherits the same normality assumption as Parametric VaR, applied even further into the tail — exactly where real returns (especially fat-tailed assets like VNQ, VWO) deviate most from normal. We rely on Historical CVaR instead, which reads the real empirical tail.

Tracking Error & Beta

Tracking Error is the volatility of the *difference* between portfolio and benchmark returns — how closely a portfolio follows what it's compared against. Beta measures sensitivity to benchmark moves: a 1% move in the benchmark corresponds to a $\beta\%$ move in the portfolio, on average.

Concentration Risk (HHI)

The Herfindahl-Hirschman Index sums squared weights. Higher HHI = more concentrated = more idiosyncratic risk tied to a few positions rather than spread across many.

Part 2 — Math Walkthrough

$$\sigma_{\text{downside}} = \sqrt{(1/n) \sum_{R_t < 0} R_t^2}$$

Worked example: returns of +4%, +5%, -1%, +3%, -6%, +2% → only -1% and -6% count. $\sigma_{\text{downside}} = \sqrt{((0.0001+0.0036)/6)} \approx 2.48\%$.

$$\text{MDD} = \min_t [(V_t - \text{Peak}_t) / \text{Peak}_t]$$

Worked example: values \$100,\$110,\$105,\$90,\$95,\$120,\$80 → running peak tracked at each step → worst drawdown is day 7: $(80-120)/120 = -33.3\%$, even though day 4 looked worse at the time (peak had grown by day 7).

Historical $\text{VaR}_{95\%} = -(5\text{th percentile of historical returns})$

Parametric $\text{VaR} = -(\mu + z \cdot \sigma)$, $z = -1.645$ for 95%

Real result (equal-weight portfolio): at 95% confidence, Parametric VaR (1.21%) was slightly higher than Historical VaR (1.07%). At 99%, this flipped: Historical (2.12%) exceeded Parametric (1.72%). This crossover is itself meaningful — the normal-distribution assumption holds up reasonably at the edge of the tail (95%) but breaks down deeper into it (99%), exactly where fat tails matter most.

$\text{CVaR}_\alpha = \text{mean of all returns} \leq \text{VaR}_\alpha \text{ threshold}$

$\beta = \text{Cov}(R_p, R_b) / \text{Var}(R_b)$

Real result: the equal-weight 9-asset portfolio's beta vs. SPY was 0.545 — it moves at roughly half of SPY's sensitivity, direct quantified proof of reduced systematic risk from diversification. Note: beta only captures *systematic* risk; total volatility also includes idiosyncratic risk (gold shocks, REIT-specific events) that beta is blind to.

$\text{HHI} = \sum w_i^2$

For n equally-weighted assets, $\text{HHI} = 1/n$ exactly — the theoretical minimum (most diversified) for that asset count.

Part 3 — Code Explanation

src/risk/risk_engine.py

```
def maximum_drawdown(cumulative_returns):
    wealth_index = 1 + cumulative_returns
    running_peak = wealth_index.cummax()
    drawdown = (wealth_index - running_peak) / running_peak
    max_dd = drawdown.min()
    trough_date = drawdown.idxmin()
    peak_date = wealth_index[:trough_date].idxmax()
```

`.cummax()` computes the running maximum up to each point — the "Peak" in the drawdown formula.

`wealth_index[:trough_date].idxmax()` deliberately finds the peak *before* the worst trough, not the all-time peak — correctly handling cases where a new peak formed after an earlier dip but before the real crash.

```
def historical_var(returns, confidence_level=0.95):
    percentile = (1 - confidence_level) * 100
    var = np.percentile(returns, percentile)
    return -var # expressed as a positive loss number
```

```
def beta(portfolio_returns, benchmark_returns):
    covariance = np.cov(portfolio_returns, benchmark_returns)[0, 1]
    benchmark_variance = np.var(benchmark_returns, ddof=1)
    return covariance / benchmark_variance
```

`np.cov()` on two series returns a 2x2 matrix; `[0, 1]` is the actual cross-covariance term beta's formula needs, not either series' own variance. `ddof=1` uses the sample ($n-1$) convention, standard for real data rather than a full population.

Module 10 — Performance Analytics

Return alone is meaningless without knowing what risk was taken to earn it. Every metric here pairs return with a different risk lens from Module 9.

Part 1 — Concepts

- **CAGR** — the standard investment-industry growth metric, using calendar years rather than trading-day counts.
- **Sortino Ratio** — Sharpe's downside-only counterpart. Doesn't penalize upside variability.
- **Calmar Ratio** — return relative to the single worst historical drawdown, not a smoothed statistical measure. Common in hedge fund/CTA reporting where return distributions can be unusual.
- **Information Ratio** — excess return over a *benchmark* (not the risk-free rate) per unit of tracking error. The standard metric for judging active management skill.

Benchmark caveat: SPY is used for Information Ratio here for consistency with earlier modules, but SPY is a pure-equity benchmark being applied to a deliberately diversified multi-asset portfolio. A negative IR here mainly reflects "this isn't SPY," not "this portfolio underperformed its actual goal." Module 15 introduces a more appropriate benchmark.

Part 2 — Math Walkthrough

$$\text{CAGR} = (V_{\text{end}} / V_{\text{start}})^{1/n_{\text{years}}} - 1$$

Worked example: \$100 → \$250 over 8 years: $\text{CAGR} = 2.5^{0.125} - 1 \approx 12.1\%$.

$$\text{Sortino} = (E(R_p) - R_f) / \sigma_{\text{downside}}$$

$$\text{Calmar} = \text{Annualized Return} / |\text{Max Drawdown}|$$

$$\text{Information Ratio} = (E(R_p) - E(R_b)) / \text{Tracking Error}$$

Real results (equal-weight portfolio):

CAGR: 7.64% (vs. 6.51% "annualized return" from Module 6 — the gap comes from two different, both-valid time conventions: calendar days/365.25 for CAGR vs. trading days/252 for the earlier figure)

Sortino: 0.530 > Sharpe: 0.377 — confirms theory directly: removing the "penalty" for upside variability (Sortino's denominator is smaller) makes risk-adjusted return look more favorable

Calmar: 0.190 — a comparatively harsh number, since it divides by the single worst historical drawdown (−33.87% in 2008), a much less forgiving denominator than smoothed volatility

Information Ratio vs SPY: −0.473 — expected, not a red flag, given the portfolio deliberately isn't 100% equities

Part 3 — Code Explanation

src/performance/performance_analytics.py

```
def calculate_cagr(cumulative_returns):
    wealth_index = 1 + cumulative_returns
    v_start, v_end = wealth_index.iloc[0], wealth_index.iloc[-1]
    n_days = (cumulative_returns.index[-1] - cumulative_returns.index[0]).days
    n_years = n_days / 365.25
    return (v_end / v_start) ** (1 / n_years) - 1
```

Uses *calendar* time (365.25 days, accounting for leap years) rather than trading-day counts — deliberately different from the 252-day convention used elsewhere, because CAGR is defined over actual elapsed years, not market-open days.

All four performance functions deliberately take *pre-computed* risk-engine outputs (downside vol, max drawdown, tracking error) as inputs rather than recalculating them — a clean separation of concerns: Module 9 computes risk, Module 10 only combines it with return.

Module 11 — Portfolio Construction

Equal-weighting is just one of several distinct construction philosophies. This module formalizes three, each with different tradeoffs between simplicity, data-reliance, and judgment.

Part 1 — Concepts

Strategy	Uses Data?	Advantage	Limitation
Equal Weight	No	No estimation error at all	Ignores every asset's risk/return/correlation
Custom Allocation	Judgment only	Encodes strategic intent	Inherently subjective, no "correct" version
Inverse Volatility	Standalone volatility only	Reacts to real risk data, simple	Ignores correlation entirely

Part 2 — Math Walkthrough

$$w_i = (1/\sigma_i) / \sum_j (1/\sigma_j)$$

Give less capital to assets that swing around more — without needing the full covariance matrix.

Real result: Inverse Volatility gave SHY (lowest-vol asset) 58.3% of the portfolio and VNQ (a volatile alternative) only 2.95% — despite VNQ's genuine diversification value seen in Module 7's correlation heatmap. This exposed the exact limitation named above: the method is correlation-blind, so it can't recognize that a volatile asset might still deserve weight because of how it interacts with everything else.

Part 3 — Code Explanation

src/portfolio/construction.py

```
def custom_allocation_portfolio(weights_dict):
    weights = pd.Series(weights_dict)
    if not np.isclose(weights.sum(), 1.0, atol=1e-6):
        raise ValueError(f"Weights must sum to 1.0, got {weights.sum():.6f}")
```

`np.isclose` rather than exact `==` : floating-point arithmetic can produce tiny rounding artifacts (0.999999999 instead of 1.0). A small tolerance avoids rejecting valid weights while still catching genuinely wrong sets.

```
def inverse_volatility_portfolio(returns, trading_days_per_year=252):
    annualized_vol = returns.std() * np.sqrt(trading_days_per_year)
    inverse_vol = 1 / annualized_vol
    return inverse_vol / inverse_vol.sum()
```

All three construction functions deliberately share one interface — return a `pd.Series` of weights indexed by ticker — so they can be compared directly and interchangeably in Module 14, with no special-case handling needed.

Module 12 — Mean-Variance Optimization

The mathematical centerpiece of the project: instead of applying a fixed rule, we search across every possible weight combination for the one that best satisfies a stated objective, using the full covariance matrix.

Part 1 — Concepts

What optimization actually is

Three parts: a **decision variable** (the weight vector), an **objective function** (minimize variance, or maximize Sharpe), and **constraints** (weights sum to 1; long-only, no shorting).

Minimum Variance Portfolio

The absolute lowest-risk point achievable given the asset universe, with zero regard for return.

Maximum Sharpe (Tangency) Portfolio

The single portfolio with the best risk-adjusted return on the entire frontier. Requires *expected returns* as an input — and future returns are fundamentally unknowable, so this method is far more sensitive to estimation error than Minimum Variance.

The Efficient Frontier

Not one optimization but many: sweep a range of target returns, and for each one solve "minimize variance subject to hitting at least this return." The resulting curve traces the full efficient set.

Why implement three ways (SciPy, then CVXPY, then PyPortfolioOpt)

SciPy is a general-purpose numerical optimizer — nothing hidden. CVXPY is a convex-optimization-specific solver, more efficient for this well-behaved class of problem. PyPortfolioOpt is a high-level library used last, as a validated convenience layer rather than a blind black box — if its answer doesn't match our own, we know something is wrong.

Part 2 — Math Walkthrough

$$\min_w w^T \Sigma w \quad \text{s.t. } \sum w_i = 1, w_i \geq 0$$

Real result — unconstrained Minimum Variance: 98.03% SHY, everything else ~0%. SHY's volatility is so much lower than every other asset that no amount of correlation cleverness elsewhere can compete with just holding it directly — a real, well-documented "corner solution" in naive MVO, not a bug.

$$\max_w (w^T \mu - R_f) / \sqrt{w^T \Sigma w} \quad \text{s.t. } \sum w_i = 1, w_i \geq 0$$

Real result — unconstrained Maximum Sharpe: QQQ 42%, IEF 36%, GLD 22%, everything else exactly 0% — including SPY. SPY was zeroed out because it's highly correlated with QQQ (0.92) but has a lower historical return, so the optimizer treated it as redundant. This is the estimation-error problem made concrete: a noisy historical average return was treated as a precise, certain input.

$$\text{Frontier point: } \min_w w^T \Sigma w \quad \text{s.t. } \sum w_i = 1, w_i \geq 0, w^T \mu = \text{target}$$

Swept across 50 target returns from the lowest to highest individual asset return. The very first few points can dip in volatility as return rises slightly — the true Minimum Variance point isn't always at the extreme low end of the sweep, since a small blend can have lower

variance than the single calmest asset alone. This dipping segment is technically feasible but "inefficient" — dominated by other points on the true frontier.

Part 3 — Code Explanation

src/optimization/mean_variance.py

```
def minimum_variance_portfolio(cov_matrix):
    initial_guess = np.array([1/n] * n)
    bounds = tuple((0, 1) for _ in range(n))
    constraints = ({'type': 'eq', 'fun': lambda w: np.sum(w) - 1},)
    result = minimize(_portfolio_variance, initial_guess,
                      args=(cov_matrix.values,), method='SLSQP',
                      bounds=bounds, constraints=constraints)
```

`bounds` enforces long-only ($0 \leq w_i \leq 1$) as a per-variable constraint. The `constraints` dict enforces "weights sum to 1" as an *equality* constraint — SciPy's convention is that the function must equal exactly zero at the solution, so `sum(w) - 1` hits zero precisely when weights sum to 1. `SLSQP` (Sequential Least Squares Programming) is SciPy's standard algorithm for constrained problems like this. `args` passes the fixed covariance matrix alongside the candidate weights SciPy generates internally at each iteration.

```
if not result.success:
    raise RuntimeError(f"Optimization failed: {result.message}")
```

Deliberate "fail loud, not silent" check — numerical optimizers can fail to converge; we want to know immediately rather than silently return a nonsensical weight vector.

Module 13 — Portfolio Constraints

Module 12 proved that unconstrained optimization can produce extreme, unusable corner solutions. This module adds realistic rules that keep the optimizer's genuine insight while preventing degenerate results.

Part 1 — Concepts

- **Maximum weight per asset** — directly prevents SHY-98%-style concentration.
- **Minimum weight per asset** — a uniform floor; note a true "0% or at least 3%" rule needs mixed-integer programming, beyond SLSQP's scope. What we implement is simpler: every held asset must be $\geq$ some floor, no exceptions.
- **Asset-class constraints** — bound whole buckets (e.g. Equity 40–70%) so strategic judgment and mathematical optimization work together.
- **Maximum HHI** — caps concentration using the same Herfindahl index from Module 9, directly as a constraint rather than just a diagnostic.
- **Target volatility** — instead of minimizing/maximizing an objective outright, hit a specific risk level exactly, then maximize return within it.

Part 2 — Math Walkthrough

SciPy inequality constraints require the function to be ≥ 0 . To express "equity weight between 40% and 70%":

$$\sum w_{\text{equity}} - 0.40 \geq 0 \quad \text{and} \quad 0.70 - \sum w_{\text{equity}} \geq 0$$

Real result — Minimum Variance with max_weight=30%: SHY and IEF both hit the 30% ceiling exactly — proof the optimizer still wants more of both, just capped. SPY reappeared (18.4%) to fill the remaining space, and TLT/GLD picked up meaningful weight too. Note: capping the low-risk side didn't force weight into growth assets — QQQ/VEA/VWO/VNQ stayed near zero, since capping one side of the risk spectrum only forces reallocation among what's left uncapped, not automatic diversification everywhere.

Real result — Minimum Variance with asset-class bounds (Equity 40-70%, FI 20-40%, Alt 5-20%): all three buckets landed exactly on their binding boundary — Equity 40.00% (floor), FI 40.00% (ceiling), Alternatives 20.00% (ceiling). This tells us the pure variance-minimizing objective wants as little equity and as much bonds/alternatives as allowed — the constraints are actively binding, not just cosmetic.

Part 3 — Code Explanation

src/optimization/mean_variance.py — shared constraint builder

```
for asset_class, (min_w, max_w) in asset_class_bounds.items():
    class_indices = [i for i, t in enumerate(tickers) if asset_class_map[t] == asset_class]
    constraints.append({"type": "ineq",
                       "fun": lambda w, idx=class_indices, lo=min_w: np.sum(w[idx]) - lo})
    constraints.append({"type": "ineq",
                       "fun": lambda w, idx=class_indices, hi=max_w: hi - np.sum(w[idx])})
```

Critical Python gotcha, handled here: `idx=class_indices` and `lo=min_w` as default arguments force each lambda to capture the *current* loop value at creation time. Without this, every lambda would share the same variable reference, and by the time SciPy actually calls them later, they'd all silently use only the *last* asset class's values — a classic Python closure bug.

```
if max_hhi is not None:
    constraints.append({"type": "ineq",
                       "fun": lambda w, cap=max_hhi: cap - np.sum(w ** 2)})
```

Directly reuses the HHI formula from Module 9 ($\sum w_i^2$) as a live constraint rather than a post-hoc diagnostic.

Module 14 — Advanced Portfolio Optimization

Two more construction philosophies — Risk Parity and Maximum Diversification — plus a full, honest comparison across every method built since Module 11.

Part 1 — Concepts

Risk Parity

Goal: every asset contributes an *equal share* of total portfolio risk. Unlike Inverse Volatility (standalone volatility only), Risk Parity uses each asset's *marginal* contribution to portfolio risk — which depends on correlation, not just standalone volatility.

Maximum Diversification

Maximizes the Diversification Ratio: how much diversification benefit is being captured, relative to the assets' own risk levels. Always ≥ 1 ; higher means more benefit captured.

Part 2 — Math Walkthrough

$$RC_i = w_i \cdot (\Sigma w)_i / \sigma_p$$

Risk Parity solves for weights such that $RC_1=RC_2=\dots=RC_n$. With 3+ assets this generally requires numerical optimization — no closed-form formula, unlike Inverse Volatility's direct calculation.

Real result: risk contributions across all 9 assets converged to ~0.004255 each, differing only in the 6th decimal — direct numerical proof the solver found true equal risk contribution. Resulting weights: SHY still dominant (61.1%), similar in shape to Inverse Volatility (58.3%) but not dramatically different asset-by-asset — a reminder that small hand-worked examples don't always predict the net effect in a complex, real 9-asset correlation structure.

$$DR = (\Sigma w_i \sigma_i) / \sigma_p$$

Worked example (2 assets, $\sigma_A=20\%$, $\sigma_B=10\%$, $\rho=0.2$):

Equal weight (50/50): $\sigma_p=12.04\%$, $DR = 15\%/12.04\% = 1.246$

Inverse-vol weight (33.3/66.7): $\sigma_p=10.33\%$, $DR = 13.33\%/10.33\% = 1.290$ — higher DR, more diversification benefit per unit of raw weighted risk, even though this weight wasn't explicitly chosen to maximize DR.

Real result (9 assets): Maximum Diversification still concentrated heavily in SHY (71.2%), with VEA and IEF at exactly zero — proof that optimizing for a *ratio* doesn't automatically avoid corner solutions either.

Full Strategy Comparison (equal-weight benchmark = HHI 0.111)

Strategy	Return	Vol	Sharpe	Max DD	HHI
Equal Weight	6.44%	11.97%	0.371	-33.86%	0.111
Custom Allocation	7.45%	12.89%	0.423	-35.51%	0.128
Inverse Volatility	3.44%	4.33%	0.331	-13.18%	0.368
Minimum Variance	1.70%	1.44%	-0.211	-5.99%	0.961
Maximum Sharpe	9.67%	10.32%	0.743	-23.65%	0.355

Risk Parity	3.18%	3.83%	0.307	-12.72%	0.400
Max Diversification	3.01%	3.46%	0.291	-12.03%	0.526

Honest synthesis (no universal "best," per project guidelines): Maximum Sharpe wins on risk-adjusted return but concentrates in 3 of 9 assets and is most exposed to estimation error. Minimum Variance wins on capital preservation but has a *negative* Sharpe ratio — the "safest" portfolio by volatility can still underperform a risk-free asset. Only Equal Weight and Custom Allocation stay genuinely diversified by HHI. Risk Parity's Sharpe (0.307) is actually *lower* than Equal Weight's (0.371) — proof that a more mathematically sophisticated method does not automatically produce a better outcome; the result depends on the objective chosen and the data's own characteristics, not the sophistication of the method alone.

Part 3 — Code Explanation

src/optimization/advanced_optimization.py

```
def _risk_contributions(weights, cov_matrix):
    port_vol = np.sqrt(weights.T @ cov_matrix @ weights)
    marginal_contributions = cov_matrix @ weights
    return weights * marginal_contributions / port_vol

def _risk_parity_objective(weights, cov_matrix):
    rc = _risk_contributions(weights, cov_matrix)
    target = rc.mean()
    return np.sum((rc - target) ** 2)
```

`cov_matrix @ weights` computes $(\Sigma \mathbf{w})_i$ for every asset in one operation — the marginal contribution vector. The objective minimizes the *dispersion* between contributions rather than targeting a fixed value directly — when every contribution equals the average exactly, this sum of squared deviations hits its theoretical minimum of zero.

```
bounds = tuple((0.0001, 1) for _ in range(n))
options={"ftol": 1e-12, "maxiter": 1000}
```

A tiny positive floor (not exactly 0) avoids numerical instability in the risk-contribution formula during the solver's intermediate steps. Tighter tolerance and more iterations than SciPy's defaults, because driving dispersion to near-zero needs more numerical precision than a simple variance minimization.

```
def _negative_diversification_ratio(weights, individual_vols, cov_matrix):
    weighted_avg_vol = weights @ individual_vols
    port_vol = np.sqrt(weights.T @ cov_matrix @ weights)
    return -(weighted_avg_vol / port_vol)
```

`individual_vols = np.sqrt(np.diag(cov_matrix))` extracts each asset's own volatility from the covariance matrix's diagonal — the same technique used for the Module 8 diversification-benefit proof.